Sandy Autonomous Control API

Complete functional API specification for Google Cloud, VM operations, Supabase, GitHub, deployment automation and Sandy customer services

Version 1.0 | 20 October 2026 | Design specification, not deployment certification

Executive mandate

Sandy MCP is a thin, typed interface to Sandy API. Sandy API owns orchestration, provider credentials, job execution, idempotency, verification and audit evidence. The executor must be able to complete routine engineering and infrastructure work end-to-end without repeated manual approvals after the owner has granted the necessary standing authorization. No API can override provider IAM, subscription limits, law, tenant consent, data ownership or safety constraints. “Full control” in this document means complete authorized administrative coverage, not an unsafe bypass of controls.

1. Scope, assumptions and target architecture

- Target Supabase project: zmqlyygzqoidkcwdysok. Sandy application ecosystem includes Microsoft Graph email, Composio integrations, calendar, WhatsApp, voice/video, agenda/tasks, memory, chat, missions, controller, billing and multi-tenant customer services.
- Google Cloud scope: projects, IAM, networking, Compute Engine, persistent disks, images, snapshots, instance templates, MIGs, load balancers, DNS, certificates, Cloud Run, GKE if adopted, Artifact Registry, Secret Manager, Cloud Storage, logging/monitoring, schedulers, budget controls and all required supporting services.
- GitHub scope: organizations and repositories, branches/commits/tags, pull requests, settings, actions, releases, deployments, webhooks, secrets, teams and issue management.
- Document is a required-operation catalogue. Proposed Sandy endpoints and operation names below are NOT claims that those endpoints already exist. Provider API examples are illustrative; implementation must be generated from current provider OpenAPI/discovery schemas.

2. Separation of duties and execution model

Required path: ChatGPT → Sandy MCP (operations.list/describe/execute/status) → Sandy API gateway → AuthN/authZ + policy engine → durable orchestration queue → typed provider adapters → verifier → append-only audit/evidence store → status/event callbacks. Direct access from Sandy MCP to unrelated Supabase/GitHub/GCP tools is not needed for Sandy administration.

- Standing authorization: owner can configure fully automatic execution of scoped engineering workflows, including production deployment. Owner approval is granted once through account/project roles, not requested again on every operation.
- No unconditional “god token”: short-lived workload identity, impersonation, least privilege, organization controls, environment-specific privileges, secret vault, mandatory tenant boundaries, immutable evidence, and out-of-band recovery access.
- Changes that irreversibly delete customer data, transfer ownership, disable all authentication/auditing, exfiltrate secrets or cross customer boundaries remain protected by explicit tenant-authorized policy, retention and provider requirements. Preauthorized production operations can still run without interactive prompts within that policy.

- Provider-denied operations must fail truthfully with the precise blocked permission, resource and remediation, never report “submitted” as “completed”.

3. Canonical API design and execution contract

API	Purpose
GET /v1/capabilities	List executable operations with availability, scope, identity and verification
GET /v1/capabilities/{operation}	Typed JSON schema, handler, required grants and example invocation
POST /v1/operations/{operation}:execute	Submit synchronous or durable async execution with inputs
GET /v1/jobs/{jobId}	Current lifecycle, provider request IDs, result and blockers
GET /v1/jobs/{jobId}/events	Streaming progress and structured step events
POST /v1/jobs/{jobId}:cancel	Cancel eligible queued/running job
POST /v1/jobs/{jobId}:retry	Retry failed idempotent steps only
POST /v1/jobs/{jobId}:rollback	Execute verified compensating plan when feasible
GET /v1/audit?resource=...	Tamper-evident chain of requested and actual changes
POST /v1/plans:validate	Dry run, permissions check, cost and blast-radius estimate
POST /v1/plans:execute	Execute a multi-service dependency graph atomically where possible

Uniform envelope: request_id, actor_id, tenant_id, environment, operation, provider_account, target_resource, params, idempotency_key, preconditions, dry_run=false, correlation_id, requested_at. Reply: accepted, job_id, state [queued|running|blocked|failed|succeeded|rolled_back|partially_succeeded], steps, provider_request_ids, observed_state, evidence, errors, timestamps, rollback_ability. A 202 response never means success.

4. Google Cloud operation catalogue

4.1 Organizations, projects, access and billing

Proposed Sandy operation(s)	Required capability
gcp.organizations.list/get; gcp.folders.list/create/move/delete	Discover organization/folder hierarchy and lifecycle.
gcp.projects.list/get/create/update/move/delete/undelete	Fully manage authorized projects; enable services and retrieve long-running operation status.
gcp.services.list/enable/disable; gcp.quotas.get/request	Manage service usage and quota changes subject to GCP decisions.
gcp.iam.policy.get/set; gcp.iam.roles.list/create/update/delete	Manage IAM bindings and custom roles with etags/conditional bindings.
gcp.serviceAccounts.list/create/update/delete; gcp.serviceAccounts.keys.rotate	Manage service identities and credentials; prefer workload federation over keys.
gcp.workloadIdentity.configure; gcp.osLogin.grant/revoke	GitHub/Sandy federation, per-VM SSH role access, OS Login.
gcp.billing.accounts.list; gcp.billing.projects.link/unlink; gcp.budgets.manage	Billing association, budget alerts and cost visibility.

4.2 Compute Engine and guest OS

Proposed Sandy operation(s)	Required capability
gcp.compute.instances.list/get/create/delete	Provision Linux/Windows instances with machine, image, network, SA, disks and metadata.
gcp.compute.instances.start/stop/reset/suspend/resume	Manage lifecycle and observe actual guest/Compute readiness.
gcp.compute.instances.resize/setMachineType/setLabels/	Modify allocated resources and authorized bootstrap metadata.

setTags/setMetadata	
gcp.compute.instances.attachDisk/detachDisk; gcp.disks.create/resize/delete	Storage management, encryption and safe filesystem resize sequencing.
gcp.compute.images.create/list/delete; gcp.snapshots.create/list/restore/delete	OS image, backup, cloning and restore operations.
gcp.compute.instances.serialOutput; gcp.compute.operations.get/wait	Boot diagnostics and real provider task completion.
gcp.vm.access.bootstrap/rotate/revoke; gcp.vm.ssh.exec; gcp.vm.winrm.exec	Remote guest commands via OS Login/IAP/managed guest agent with explicit caller identity.
gcp.vm.files.upload/download; gcp.vm.packages.install/update/remove	Upload artifacts, install software, patch OS and run PowerShell/shell scripts.
gcp.vm.services.start/stop/restart/status; gcp.vm.reboot; gcp.vm.firewallGuest.manage	Guest service management with health proof and reboot handling.
gcp.vm.agent.install/update/status; gcp.vm.bootstrap.getLogs	Agent enrollment, privilege configuration, signed command replay defense.
gcp.compute.instanceGroups.*; gcp.compute.templates.*	Autoscaling groups, instance template release and rolling update.

4.3 Network, hosting, platform and operations

Proposed Sandy operation(s)	Required capability
gcp.vpc.networks.*; gcp.vpc.subnets.*; gcp.vpc.routes.*	Provision and manage VPCs, private networks and routes.
gcp.firewalls.*; gcp.networkPolicies.*; gcp.nat.*; gcp.vpn.*	Ingress/egress policy, NAT and interconnection.
gcp.addresses.*; gcp.dns.zones.*; gcp.dns.records.*	Static IP, public/private DNS, DNS propagation verification.
gcp.loadBalancers.*; gcp.certificates.*; gcp.healthChecks.*	HTTP(S)/TCP LB, TLS, backend services and health checks.
gcp.cloudRun.services.*; gcp.cloudRun.jobs.*; gcp.gke.*	Deploy/manage Cloud Run and optional Kubernetes workloads.
gcp.artifacts.repositories.*; gcp.storage.buckets.*; gcp.storage.objects.*	Build artifacts, object storage, lifecycle, access.
gcp.secrets.createVersion/access/rotate/revoke	Secret lifecycle with redaction; secret values never included in job logs.
gcp.logging.query/sinks.manage; gcp.monitoring.metrics/alerts.*	Logs, metrics, alert policies, synthetic probes and incident tracking.
gcp.scheduler.jobs.*; gcp.pubsub.topics/subscriptions.*; gcp.tasks.queues.*	Scheduled jobs, events and resilient async automation.
gcp.security.findings.list; gcp.kms.keys.*; gcp.backups.*	Security observability, encryption keys, backups, disaster recovery.

5. Supabase platform and data plane catalogue

5.1 Supabase project management and deployments

Proposed Sandy operation(s)	Required capability
supabase.orgs.list; supabase.projects.list/get/create/update/pause/restore/delete	Authorized organization/project control and platform status.
supabase.project.config.get/set; supabase.network.*; supabase.domains.*	API/Auth/network/database/project settings, custom domains where supported.
supabase.branches.create/list/get/merge/delete; supabase.db.diff	Preview environments and schema comparison.
supabase.db.migrations.plan/apply/list/repair/verify	Versioned migrations, locks, checksums, drift and smoke test.
supabase.functions.list/get/deploy/delete/invoke/logs	Manage all Edge Functions including sandy-api, webhooks, worker and calendar.
supabase.functions.secrets.list/set/delete/rotate	Secure, audited function configuration.
supabase.project.logs.query; supabase.project.health;	Monitoring, PITR restore where provisioned, disaster recovery.

supabase.backups.list/restore	

5.2 PostgreSQL administrative coverage

Proposed Sandy operation(s)	Required capability
supabase.sql.query; supabase.sql.execute; supabase.sql.transaction	Read and write authorized SQL, parameterized statements and transactions.
supabase.schema.tables/views/materializedViews.*	Create/alter/drop data structures via migration.
supabase.schema.functions/procedures/triggers.*	Create/replace database logic, RPC, scheduled procedures.
supabase.schema.indexes/constraints/types/extensions.*	All schema objects, extensions where provider supports.
supabase.database.roles/grants/defaultPrivileges.*	Postgres roles, privileges and separation by tenant.
supabase.database.rls.policies.*; supabase.database.replication.*	RLS, publications, logical replication and CDC.
supabase.data.insert/upsert/update/delete/bulkImport/ bulkExport	Data plane CRUD, bulk migration and bounded exports.
supabase.database.inspect/explain/analyze/vacuum/reindex	Diagnostics, performance, lock management and maintenance.
supabase.database.connections/pooler/slowQueries/locks	Connections, performance monitoring and intervention.
supabase.database.backup/snapshot/restore/verify	Recoverability, retention and backup integrity.
supabase.database.cron/jobs.*; supabase.database.webhooks.*	Automated DB work and event propagation.

5.3 Supabase built-in customer infrastructure

Proposed Sandy operation(s)	Required capability
supabase.auth.users.list/get/create/update/disable/delete	Auth user administration and account recovery within tenant policy.
supabase.auth.providers/config/hooks/sessions/mfa.*	Identity configuration, MFA, SSO and token revocation.
supabase.storage.buckets.*; supabase.storage.objects.*; supabase.storage.policies.*	File content, access policy, signed URLs, retention and purge.
supabase.realtime.channels/permissions/publications.*	Realtime policies, broadcasts and subscriptions.
supabase.vault.secrets.*; supabase.logs.query	Secrets and audit/operations logs.
supabase.tenant.export/erase/restore; supabase.compliance.audit	Tenant-bound export, deletion and legal hold enforcement.

6. GitHub administration and CI/CD catalogue

6.1 Git objects and collaboration

Proposed Sandy operation(s)	Required capability
github.apps.installations.list/verify; github.repos.list/get/create/update/archive/delete	App installations and repository lifecycle.
github.refs.list/create/update/delete; github.branches.list/protect	Branch pointers and controls within assigned repository policy.
github.files.get/create/update/delete; github.git.blobs/trees/commits.*	Single-file or atomic multi-file code changes and git object manipulation.
github.pullRequests.list/create/update/review/merge/close	PR-based code delivery and traceable reviews.
github.issues.*; github.labels.*; github.milestones.*; github.discussions.*	Engineering coordination, backlog and releases.
github.tags.*; github.releases.create/publish/delete; github.packages.*	Versioning and distributable releases.

6.2 Repository automation and administration

Proposed Sandy operation(s)	Required capability
github.actions.workflows.list/dispatch/enable/disable; github.actions.runs.get/cancel/rerun	CI/CD execution and verified workflow conclusion.
github.actions.artifacts.list/download/delete; github.checks.*; github.statuses.*	Artifact collection, build test results and commit status.
github.deployments.create/status; github.environments.list/update	Environment deployments and deployment state.
github.repoSecrets.set/remove; github.variables.set/remove; github.environmentSecrets.*	CI variables and encrypted secret management.
github.webhooks.list/create/update/delete/deliveries/ redeliver	Reliable event pipelines and signature verification.
github.collaborators/teams/permissions.*; github.rulesets.*	Manage repo collaborators, authorized roles and applicable rule sets.
github.audit.list; github.security.alerts/list; github.codeScanning.*	Security scanning, audit and remediation tracking.

7. Sandy customer-service operations: complete domain catalogue

7.1 Tenancy & subscription

Required operation family: sandy.tenancy_and_subscription.*

Coverage: customers list/get/create/update/suspend/reactivate; organizations create/edit; invitations; roles; seats; feature flags; quotas; plan entitlements; invoice and subscription status; lifecycle/offboarding. All create/update/delete actions require tenant-scoped authorization, idempotency, before/after audit and live postcondition verification. Every read/list includes pagination/filter/search where meaningful.

7.2 Authentication & security

Required operation family: sandy.authentication_and_security.*

Coverage: account/profile; sessions/revoke; passkeys; MFA; recovery; OAuth connections; consent states; permissions; device management; access logs; tenant policies. All create/update/delete actions require tenant-scoped authorization, idempotency, before/after audit and live postcondition verification. Every read/list includes pagination/filter/search where meaningful.

7.3 Connections/Composio

Required operation family: sandy.connections_composio.*

Coverage: connections discover/connect/refresh/health/reconnect/disconnect; discover provider tools; execute authenticated read/write; scopes; provider webhook lifecycle; per-company assignments. All create/update/delete actions require tenant-scoped authorization, idempotency, before/after audit and live postcondition verification. Every read/list includes pagination/filter/search where meaningful.

7.4 Email & Microsoft Graph

Required operation family: sandy.email_and_microsoft_graph.*

Coverage: mailboxes discover; folders and messages list/read/search; attachment download/upload; draft/create/update; send/reply/replyAll/forward; categorise/archive/delete; mail rules; webhook subscribe/renew; delta sync; delivery reconciliation. All create/update/delete actions require tenant-scoped authorization, idempotency, before/after audit and live postcondition verification. Every read/list includes pagination/filter/search where meaningful.

7.5 Google/Gmail

Required operation family: sandy.google_gmail.*

Coverage: Gmail thread/search/read/send/labels; Google Drive CRUD/search/permissions; Calendar events/attendees; Contacts search/edit; provider-specific sync tokens and push notifications. All create/update/delete actions require tenant-scoped authorization, idempotency, before/after audit and live postcondition verification. Every read/list includes pagination/filter/search where meaningful.

7.6 Calendar & scheduling

Required operation family: sandy.calendar_and_scheduling.*

Coverage: calendars list/connect; event CRUD; recurring series; attendees; availability; timezones; conflicts; attachments; reminders; Google/Outlook/iCal bidirectional incremental sync; subscription renewals. All create/update/delete actions require tenant-scoped authorization, idempotency, before/after audit and live postcondition verification. Every read/list includes pagination/filter/search where meaningful.

7.7 WhatsApp

Required operation family: sandy.whatsapp.*

Coverage: accounts/numbers/templates status; inbound/outbound message; media upload/download; webhook signature/dedup; contact opt-in; template sends; delivery/read receipts; window checks; call session where available. All create/update/delete actions require tenant-scoped authorization, idempotency, before/after audit and live postcondition verification. Every read/list includes pagination/filter/search where meaningful.

7.8 Voice, SIP & video

Required operation family: sandy.voice_sip_and_video.*

Coverage: conference create/join/end; SIP trunk config; rooms/participants; tokens; recording policies; transcripts; callbacks; audio/video bridge status; VM deployment and media health. All create/update/delete actions require tenant-scoped authorization, idempotency, before/after audit and live postcondition verification. Every read/list includes pagination/filter/search where meaningful.

7.9 Web chat & conversations

Required operation family: sandy.web_chat_and_conversations.*

Coverage: conversation create/search/read/archive; message send/edit; channels link; streaming; attachments; tools dispatch; cross-channel threading; transcripts and activity chronology. All create/update/delete actions require tenant-scoped authorization, idempotency, before/after audit and live postcondition verification. Every read/list includes pagination/filter/search where meaningful.

7.10 Memory & social graph

Required operation family: sandy.memory_and_social_graph.*

Coverage: facts extract/store/revise/reject; biography; evidence; entities/relationships; provenance; privacy scopes; dedup; search; erase/export; knowledge graphs. All create/update/delete actions require tenant-scoped authorization, idempotency, before/after audit and live postcondition verification. Every read/list includes pagination/filter/search where meaningful.

7.11 Agenda, tasks & projects

Required operation family: sandy.agenda_tasks_and_projects.*

Coverage: task/tree CRUD; subtasks; reorder; deadlines; reminders; priority; company/category; delegation; sharing; comments; attachments; completion; approval queue; dependencies. All create/update/delete actions require tenant-scoped authorization, idempotency, before/after audit and live postcondition verification. Every read/list includes pagination/filter/search where meaningful.

7.12 Missions & autonomous work

Required operation family: sandy.missions_and_autonomous_work.*

Coverage: missions create/plan/execute/pause/resume/cancel; tasks assign; approvals where tenant-required; progress; artifacts; error/retry; acceptance verification. All create/update/delete actions require tenant-scoped authorization, idempotency, before/after audit and live postcondition verification. Every read/list includes pagination/filter/search where meaningful.

7.13 Controller & QA

Required operation family: sandy.controller_and_qa.*

Coverage: dashboards; health; telemetry; errors; incidents; audit; examiner test-suite; experiment/run metrics; backlog dedup; release promotion; regression checks. All create/update/delete actions require tenant-scoped authorization, idempotency, before/after audit and live postcondition verification. Every read/list includes pagination/filter/search where meaningful.

7.14 Files & documents

Required operation family: sandy.files_and_documents.*

Coverage: document create/read/edit/version/delete; OCR/index/search; sharing; source provenance; retention; generated DOCX/PDF/XLSX/PPTX; attachments; exports. All create/update/delete actions require tenant-scoped authorization, idempotency, before/after audit and live postcondition verification. Every read/list includes pagination/filter/search where meaningful.

7.15 Notifications & alerts

Required operation family: sandy.notifications_and_alerts.*

Coverage: templates; push/email/WhatsApp notification; subscriptions; preferences; delivery attempts; retries; escalation; quiet hours; opt-outs. All create/update/delete actions require tenant-scoped authorization, idempotency, before/after audit and live postcondition verification. Every read/list includes pagination/filter/search where meaningful.

7.16 Payments and commerce

Required operation family: sandy.payments_and_commerce.*

Coverage: plans/prices; invoices; payment intents and refunds; payout visibility; taxes; entitlements; financial audit and reconciliation; PCI boundary. All create/update/delete actions require tenant-scoped authorization, idempotency, before/after audit and live postcondition verification. Every read/list includes pagination/filter/search where meaningful.

7.17 Customer lifecycle & support

Required operation family: sandy.customer_lifecycle_and_support.*

Coverage: signup/onboarding; CRM/contact history; support ticketing; consent; diagnostics; SLAs; churn; account deletion/export; data-subject requests. All create/update/delete actions require tenant-scoped authorization, idempotency, before/after audit and live postcondition verification. Every read/list includes pagination/filter/search where meaningful.

7.18 Search, agents, tools & AI

Required operation family: sandy.search_agents_tools_and_ai.*

Coverage: routing; model configurations; prompts versioning; tool catalogs; agent runtime; context sources; retrieval; usage/cost accounting; evaluations; guardrails. All create/update/delete actions require tenant-scoped authorization, idempotency, before/after audit and live postcondition verification. Every read/list includes pagination/filter/search where meaningful.

7.19 Product configuration

Required operation family: sandy.product_configuration.*

Coverage: branding/website/pages; templates; locales; currencies; feature flags; settings; frontend build/deploy; config rollback. All create/update/delete actions require tenant-scoped authorization, idempotency, before/after audit and live postcondition verification. Every read/list includes pagination/filter/search where meaningful.

8. Production deployment workflows

A. Change Sandy source code

Read live repo and target SHA → prepare patch → create atomic commit / PR or authorized direct commit → invoke CI → await tests/build artifacts → deploy to environment → run smoke tests → compare deployed commit hash → record evidence.

B. Supabase schema and functions

Introspect live schema and migration history → build reversible migration → apply to preview → test RLS/tenant isolation and compatibility → snapshot → apply versioned production migration → deploy functions/secrets → health and query verification → mark completed.

C. New Google Cloud VM/service

Preflight quotas/service enablement/IAM/network → provision VM/disk/service account/firewall → enroll managed access agent → install software → configure secrets/TLS/DNS → register service + monitoring → execute synthetic end-to-end test → capture guest and external availability.

D. Google Calendar bidirectional sync

Verify connections and scopes → create/renew provider push watch/subscriptions → receive events with signature validation → store delta cursor → incremental pull → conflict-safe upsert in both directions → dedup and echo suppression → monitor expiry and repair; assert actual changes both ways.

E. Video meeting service

Provision media/turn/sip infrastructure → bootstrap services and certificates → configure room/session signaling → test remote join, call establishment, media paths and teardown → observe latency, ports, reliability → automatic restart/recovery.

F. Incident recovery

Alert → obtain provider and application evidence → classify blast radius → safe remediation plan → carry out preauthorized fix → canary probe → rollback when postconditions fail → log root cause and prevention.

9. Authorization without per-operation manual interruption

Provide an owner-configured AUTONOMOUS_ENGINEERING standing policy for a designated Sandy service principal, project/environment and allowed domains. The service principal may plan, modify code, execute CI, migrate the database, provision/patch VMs and release services within assigned project roles. No manual approval prompts for actions already covered by the standing grant. The policy must be transparent, revocable, time-bound/rotatable, tenant-isolated and enforce provider-defined access checks. GitHub branch protection and organization rules remain effective unless the organization owner changes them through authorized settings.

Recommended consent model: single initial connection/install authorization for GCP workload identity, Supabase scoped Management API credentials and DB connection, GitHub App installation permissions, Microsoft/Google

customer OAuth per account, WhatsApp/voice provider business authorization. Customer data actions use each tenant's permissions; Sandy engineering rights do not silently confer access to unrelated customers.

10. Required non-negotiable operational guarantees

- All operations are truthful: verified=false, exposed=true, accepted=true and succeeded=true are distinct states. No fabricated deployment claims.
- Durability: transactional outbox, job leases, dead-letter queue, retry with jitter and duplicate suppression; recover crash mid-operation.
- Conflicts: if-match/etag or commit SHA preconditions; migration checksum and database advisory lock.
- Evidence: Git SHA, workflow run ID, database migration ID, GCP operation ID, VM service health, logs, timestamp, public endpoint check.
- Secrets: KMS/Secret Manager/Vault, OIDC federation, encryption, rotation, redacted logs; no tokens in user-facing errors.
- Tenant isolation: RLS, row ownership, graph/provider account mapping, user consent and precise audit provenance.
- Data recovery: backups and restore drills, forward fixes for destructive migrations, retention, and disaster-recovery runbooks.
- Resource controls: per-tenant quotas, abuse controls, budget and provider limitations must be surfaced, not bypassed.
- Observability: correlated traces, on-call notifications, SLOs, latency, delivery and queue-depth indicators.
- Test framework: integration, contract, canary and rollback tests with genuine provider evidence before declaring production ready.

11. Current observed Sandy MCP baseline (live registry checked 20 October 2026)

The Sandy operation registry returned 50 named operations, all marked verified=false. It includes calendar CRUD, read-only Composio discovery/execution, engineering change and job operations, GitHub read/commit/merge/push_files/update_file, files and memory operations. Its catalogue also advertises engineering.change.auto_approve and engineering.job.auto_approve, but their registry presence is NOT evidence of executable handling. Discovered Edge Functions were also marked verified=false and execution_enabled=false. Dedicated MCP tools separately include a read-only VM status operation and approval/queue-centered deployment tools. This strongly indicates a gap between registered capability and validated autonomous operation. A thorough implementation should test each actual handler rather than merely adding registry entries.

Gap class	Acceptance gate
Registry vs reality	Every advertised write operation executes authorized sandbox mutation and proves postcondition.
VM guest operations	Create, remote-install, restart, verify service from Sandy MCP end-to-end.
GitHub delivery	Commit → workflow → successful build → target deployment observed.
Supabase administration	Migration + function deployment + rollback rehearsal pass in intended project.
Customer services	Each domain has functional typed read/write tools with tenant-safe live tests.
Autonomy	A single owner-authorized change deploys without an additional per-job human prompt.
Failure truth	Denied or failed operation reports blocked/failed; no false

	success.

12. Recommended implementation sequence

1. Inventory current running API, identity roles, operation registry and provider installs; generate a missing-operation matrix against this catalogue.

2. Implement durable job orchestration and truthful state reporting; verify credentials and policy via preflight.

3. Finish GitHub source editing and GitHub Actions end-to-end pipeline; deploy Sandy API itself from CI.

4. Implement Supabase Management API, SQL migrations, edge function/secrets deployment and DB verification.

5. Implement Google Cloud project/IAM/compute/network + managed guest agent and remote installation/execution.

6. Implement service-level deploy, DNS/TLS, testing, backups and rollback.

7. Expose tenant-safe customer service operations with full provider connection management.

8. Run multi-provider full-stack acceptance suite through Sandy MCP alone and retain proof.

13. Acceptance test suite: minimum evidence required

Suite	Required observed result
GCP VM	Create a disposable VM, connect, install a simple service, expose a health endpoint, patch/restart, capture logs, restore snapshot, decommission; confirm every GCP operation and guest result.
GitHub	Modify an actual test file, commit, trigger workflow, inspect build outcome, deploy and prove deployed SHA.
Supabase	Create temporary table/trigger/RLS through migration, CRUD, deploy temporary edge function, confirm logs and remove using migration.
Multi-service	Change code + DB schema + VM service in one orchestration job with compensating/forward-recovery paths.
Calendar	Create/update/delete event in Google and Sandy, verify 2-way propagation, dedup and recurrence/timezone handling.
Communications	Send test email/WhatsApp where authorized; prove delivery/status and attachment presence; avoid duplicate send.
Customer isolation	Tenant A cannot access Tenant B data, connectors, files, credentials or notifications.
Autonomy	No manual prompt for change within standing authorized policy; unauthorized request denied with actionable cause.

14. External provider documentation and implementation notes

- Google Cloud REST: https://cloud.google.com/compute/docs/reference/rest/v1/
- Google Cloud OS Login: https://docs.cloud.google.com/compute/docs/oslogin/rest
- Supabase Management API: https://supabase.com/docs/reference/api/introduction
- Supabase deployment/branching: https://supabase.com/docs/guides/deployment
- Supabase for Platforms: https://supabase.com/docs/guides/integrations/supabase-for-platforms
- GitHub REST API: https://docs.github.com/en/rest
- GitHub repository contents: https://docs.github.com/en/rest/repos/contents

Completeness rule: These are capability families, not a static verbatim enumeration of every evolving provider endpoint. A production registry generator must ingest current Google discovery documents, Supabase OpenAPI/CLI capabilities, GitHub OpenAPI, and the deployed Sandy internal API, normalize permitted operations, then test and attest each one. Where provider APIs do not expose an action, use a securely enrolled, auditable agent or explicitly declare unsupported, not a fictional endpoint.

END OF SPECIFICATION